Day 173 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 6 reference

Watch: MLFlow Tutorial Part 1: Experiment Tracking (Anton T. Ruberts) | Machine Learning Experiment Tracking with VS Code, Git, and DVC (DVCorg)

`day173_experiment_tracking.py`, segment by segment, in the order `main()` uses things. Today's new parts are fingerprint, file_hash, run_id, Run, Tracker and query, copied unchanged into day173_exercise.py; Day 161's Adult loaders and Day 171's EarlyStopper, train_es and row_losses are imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: T = the tracker · R = the list of runs read back · r = one run (a dictionary), or the Run object inside a with block · p = settings · D = the data arrays · data, code = their hashes

Segment 0 — Setup

Imports and constants

CODE

```
import contextlib
import hashlib
import io
import json
import os
import platform
import sys
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(ROOT, "week-24-model-evaluation-statistics"))
with contextlib.redirect_stdout(io.StringIO()):
    import day161_exercise as d161
    import day171_exercise as d171
d126 = d171.d126
LOG = os.path.join(HERE, "day173_runs.jsonl")
N_RUNS = 40
CRASH_AT = 17
TRAIN_ROWS, STOP_ROWS = 2000, 6000
TOP, NEW_SEEDS = 5, 10
```

- `import hashlib` — the standard library's hashes; sha256 turns any bytes into a 64-hex-digit fingerprint.
- `import json` — turns dictionaries into text and back.
- `import platform` — facts about the running Python, recorded with every run.
- `with contextlib.redirect_stdout(io.StringIO()):` — the imported exercises' hand checks print into a buffer, not the screen.

Segment 1 — The parts

fingerprint, file_hash and run_id

MATH

id = sha256(text)[:12]: 48 bits; two different runs sharing an id is vanishingly unlikely at study sizes

CODE

```
def fingerprint(*arrays):
    h = hashlib.sha256()
    for a in arrays:
        a = np.ascontiguousarray(a); h.update(f"{a.shape}{a.dtype}".encode());
            h.update(a.tobytes())
    return h.hexdigest()[:12]
def file_hash(path):
    return hashlib.sha256(open(path, "rb").read()).hexdigest()[:12]
def run_id(params, seed, data, code):
    key = json.dumps(dict(params=params, seed=seed, data=data, code=code), sort_keys=True)
    return hashlib.sha256(key.encode()).hexdigest()[:12]
```

- `def fingerprint(*arrays):` — *arrays collects any number of arguments into a tuple.
- `np.ascontiguousarray(a)` — one solid block of memory, so tobytes() gives the values in a fixed order.
- `h.update(...)` — feeds more bytes into the same hash; the order matters.
- `h.hexdigest()[:12]` — the hash as hex text; the first 12 digits are kept.
- `json.dumps(..., sort_keys=True)` — keys written in sorted order, so {lr, b} and {b, lr} give the same text and the same id.

Run: a context manager

CODE

```
class Run:
    def __init__(self, tracker, params, seed, data, code):
        self.t, self.rec = tracker, dict(id=run_id(params, seed, data, code), params=params,
            seed=seed, data=data, code=code)
        self.history, self.summary = [], {}
    def log(self, step, **metrics):
        self.history.append(dict(step=step, **{k: float(v) for k, v in metrics.items()}))
    def __enter__(self):
        self.t0 = time.time()
        self.t.write(dict(event="start", time=round(self.t0, 3), env=self.t.env, **self.rec))
        return self
    def __exit__(self, kind, err, tb):
        self.t.write(dict(event="end", id=self.rec["id"], status="failed" if kind else "finished",
            error=repr(err) if kind else None, seconds=round(time.time() - self.t0, 3),
            summary=self.summary, history=self.history))
        return False
```

- `def __enter__(self):` — runs when a with block starts; what it returns is bound by `as r`.
- `def __exit__(self, kind, err, tb):` — runs when the block ends; kind, err and tb are None unless an exception escaped.
- `"failed" if kind else "finished"` — a conditional expression: one value or the other.
- `repr(err)` — the exception as text, e.g. RuntimeError('...').
- `**self.rec` — unpacks a dictionary's entries as keyword arguments, here into dict(...).
- `return False` — lets the exception carry on after it is logged, so the caller still sees the crash.

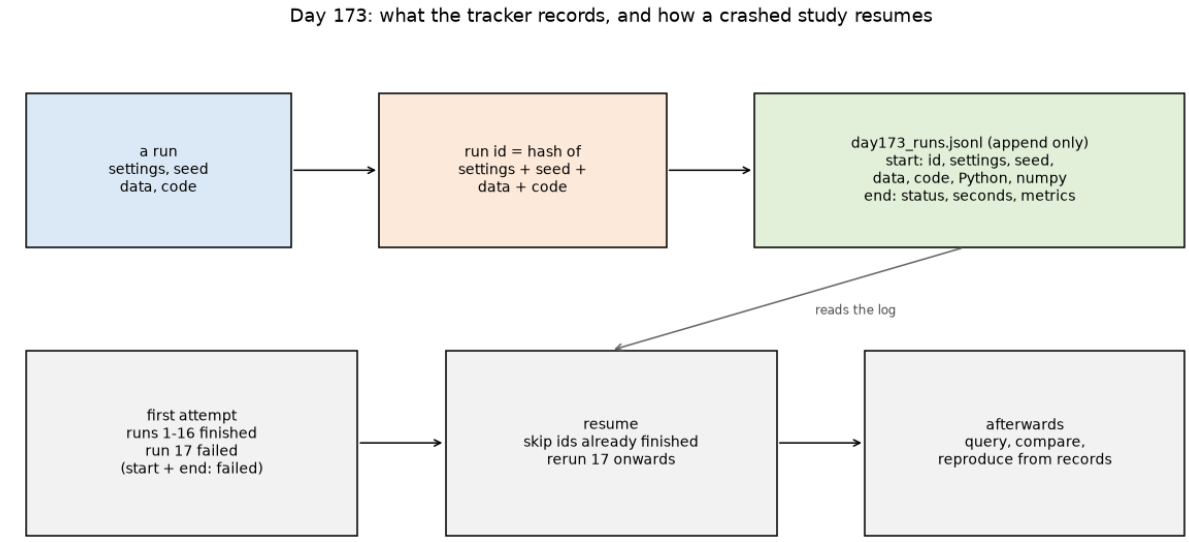
Tracker and query

CODE

```
class Tracker:
    def __init__(self, path):
        self.path = path
        self.env = dict(python=platform.python_version(), numpy=np.__version__)
    def write(self, record):
        with open(self.path, "a", encoding="utf-8") as f:
            f.write(json.dumps(record, sort_keys=True) + "\n")
    def run(self, params, seed, data, code):
        return Run(self, params, seed, data, code)
    def runs(self):
        out = {}
        if not os.path.exists(self.path):
            return []
        for line in open(self.path, encoding="utf-8"):
            r = json.loads(line)
            if r["event"] == "start":
                out[r["id"]] = dict(r, status="unfinished")
            else:
                out[r["id"]].update(status=r["status"], error=r["error"], seconds=r["seconds"],
                    summary=r["summary"], history=r["history"])
        return list(out.values())
    def done(self, rid):
        return any(r["id"] == rid and r["status"] == "finished" for r in self.runs())
def query(runs, where=lambda r: True, by=None, n=None):
    out = [r for r in runs if r["status"] == "finished" and where(r)]
    if by is not None:
        out.sort(key=lambda r: r["summary"][by])
    return out[:n] if n else out
```

- `open(self.path, "a", encoding="utf-8")` — "a" appends: writes go to the end, earlier lines stay.
- `json.dumps(record, sort_keys=True) + "\n"` — one record, one line.
- `dict(r, status="unfinished")` — a copy of r with one key added or replaced.
- `out[r["id"]].update(...)` — merges new keys into the start record held under that id.
- `any(... for r in self.runs())` — True as soon as one run matches.
- `where=lambda r: True` — a default argument that is a function: keep everything unless told otherwise.
- `out[:n] if n else out` — the first n, or the whole list when n is None.

SYSTEM DIAGRAM



What a run record holds, and how a crashed study resumes.

Segment 2 — The study

load_data, sample_configs and train_and_score

CODE

```
def load_data():
    tr, _ = d161.load_adult(); fi, va = d161.split_rows(len(tr["y"]))
    ref = {k: v[fi] for k, v in tr.items()}
    X = d161.onehot_matrix(tr, ref).astype(np.float32); y = tr["y"].astype(np.float32)
    p = np.random.RandomState(0).permutation(fi)
    return dict(Xa=X[p[:TRAIN_ROWS]], ya=y[p[:TRAIN_ROWS]], Xs=X[p[-STOP_ROWS:]],
        ys=y[p[-STOP_ROWS:]], Xv=X[va], yv=y[va])
def sample_configs(n, seed=0):
    rng = np.random.RandomState(seed)
    return [dict(lr=float(10 ** rng.uniform(-4, -1.5)), patience=int(rng.randint(1, 21)),
                 batch=int(rng.choice([32, 64, 128, 256]))) for _ in range(n)]
def train_and_score(D, p, seed, run=None):
    es = d171.EarlyStopper(p["patience"])
    net, acts, curve = d171.train_es(D["Xa"], D["ya"], D["Xs"], D["ys"], d126.Adam, p["lr"], seed,
        es, batch=p["batch"])
    val = float(d171.row_losses(net, acts, D["Xv"], D["yv"]).mean())
    if run is not None:
        for e, v in enumerate(curve):
            run.log(e + 1, stop_loss=v)
        run.summary.update(val_loss=val, best_epoch=es.best_epoch + 1, epochs=len(curve))
    return val, es.best_epoch + 1, len(curve)
```

- `p[-STOP_ROWS:]` — the last 6,000 positions of the shuffled order: the stop set.
- `10 ** rng.uniform(-4, -1.5)` — uniform in the exponent: as many small rates as large ones (log-uniform).
- `float(...), int(...)` — plain Python numbers; numpy's own types cannot be written by json.
- `for e, v in enumerate(curve):` — each epoch's number (from 0) and stop-set loss.
- `run.summary.update(val_loss=..., ...)` — adds the final numbers to the record written at the end.

study: skip what is finished, crash on purpose

CODE

```
def study(T, D, configs, data, code, crash_at=None):
    ran = skipped = 0
    for k, p in enumerate(configs):
        if T.done(run_id(p, 0, data, code)):
            skipped += 1; continue
        with T.run(p, 0, data, code) as r:
            if k + 1 == crash_at:
                raise RuntimeError(f"simulated crash in run {k + 1}")
            train_and_score(D, p, 0, r); ran += 1
    return ran, skipped
```

- `skipped += 1; continue` — two statements on one line; continue jumps to the next configuration.
- `with T.run(p, 0, data, code) as r:` — the start record is written here, before training.
- `raise RuntimeError(...)` — the simulated crash; __exit__ logs it as failed, then it propagates.

demo_study: the crash and the resume

CODE

```
try:
    study(T, D, configs, data, code, crash_at=CRASH_AT)
except RuntimeError as e:
    print(f"  first attempt stopped: {e}")
R = T.runs()
print(f"  log after the crash: {sum(r['status'] == 'finished' for r in R)} finished, "
      f"{sum(r['status'] == 'failed' for r in R)} failed ({[r['error'] for r in R if r['status']
          == 'failed'][0]})")
ran, skipped = study(T, D, configs, data, code)
```

- `try: ... except RuntimeError as e:` — catches the crash so the script can carry on and resume.
- `[...][0]` — the first (here the only) failed run's error text.

Segment 3 — Using the log

Reproducing and the seed

MATH

$$optimism = mean(new\ seeds) - logged\ seed\text{-}0\ score \qquad selection\ effect \approx optimism(top) - optimism(control)$$

CODE

```
picks = query(R, by="val_loss")[::8]
same = 0
for r in picks:
    val, best, ep = train_and_score(D, r["params"], r["seed"])
    ok = val == r["summary"]["val_loss"] and best == r["summary"]["best_epoch"] and ep ==
        r["summary"]["epochs"]
    same += ok
mid = query(R, by="val_loss")[17:22]; gaps = []
for r in mid:
    v = np.array([train_and_score(D, r["params"], s)[0] for s in range(1, NEW_SEEDS + 1)])
    gaps.append(v.mean() - r["summary"]["val_loss"])
```

- `[::8]` — every eighth item, starting with the first.
- `val == r["summary"]["val_loss"]` — exact equality of two floats: true only if every bit matches.
- `same += ok` — True counts as 1, False as 0.
- `[17:22]` — items 18 to 22 when counting from 1: the control, not chosen for a good score.

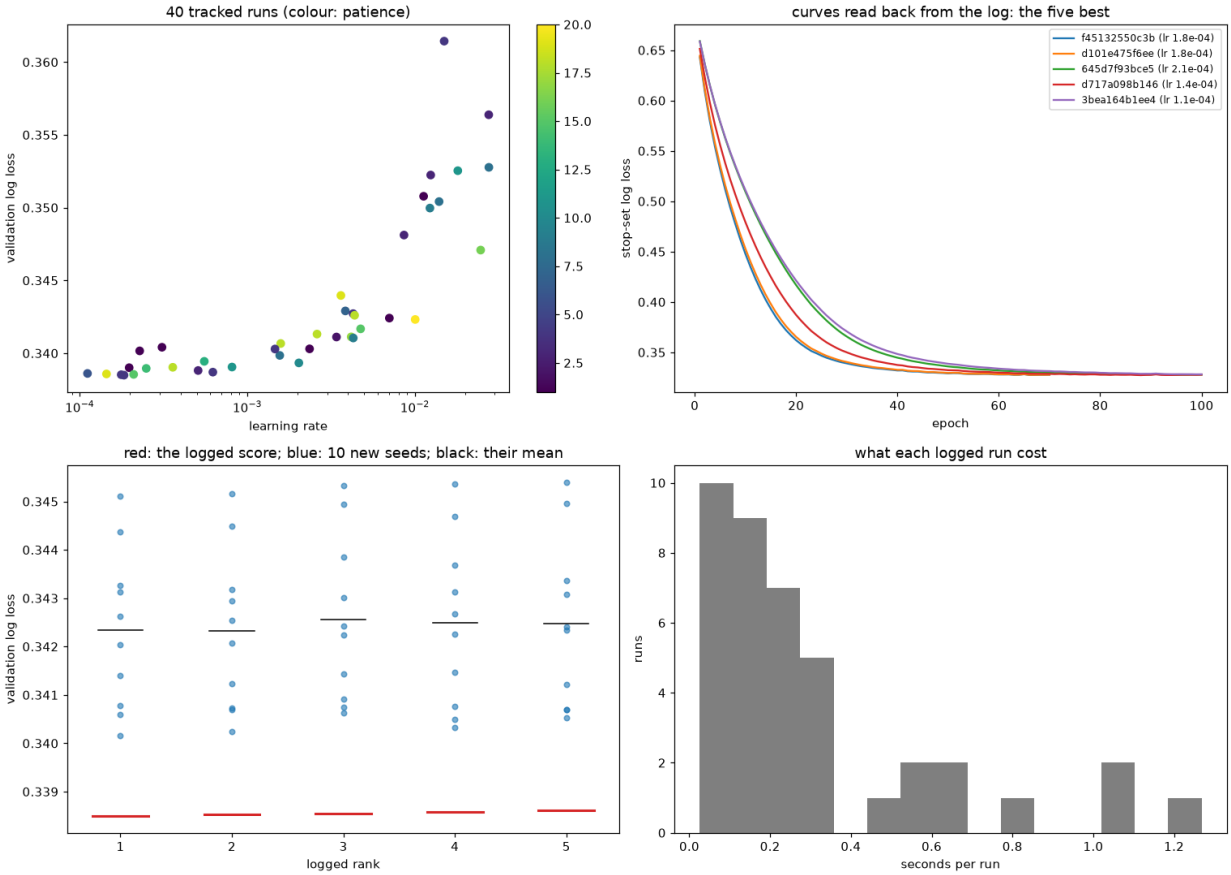
demo_fingerprints: changed data, changed code

CODE

```
D2 = dict(D); keep = np.random.RandomState(5).rand(TRAIN_ROWS) > 0.01
D2["Xa"], D2["ya"] = D["Xa"][keep], D["ya"][keep]
data2 = fingerprint(D2["Xa"], D2["ya"], D2["Xs"], D2["ys"], D2["Xv"], D2["yv"])
p = query(T.runs(), by="val_loss", n=1)[0]["params"]
with T.run(p, 0, data2, code) as r:
    train_and_score(D2, p, 0, r)
a, b = [x for x in T.runs() if x["params"] == p and x["seed"] == 0]
print(f"  {TRAIN_ROWS - keep.sum()} of {TRAIN_ROWS} training rows dropped: data {a['data']} ->
    {b['data']}; same settings and seed, "
      f"val {a['summary']['val_loss']:.4f} -> {b['summary']['val_loss']:.4f}; run ids differ:
          {a['id'] != b['id']}")
X3 = D["Xa"].copy(); X3[0, 0] += 1e-6
print(f"  one value changed by 1e-6: fingerprint {fingerprint(X3, D['ya'], D['Xs'], D['ys'],
    D['Xv'], D['yv'])} "
      f"against {data}")
src = open(__file__, "rb").read(); src2 = src.replace(b"TOP, NEW_SEEDS = 5, 10", b"TOP, NEW_SEEDS
    = 5, 11")
```

- `dict(D)` — a shallow copy: replacing D2's arrays leaves D's alone.
- `rand(TRAIN_ROWS) > 0.01` — True for about 99% of rows: those are kept.
- `a, b = [x for x in ...]` — unpacks a list that must hold exactly two runs; anything else raises an error.
- `open(__file__, "rb")` — __file__ is this script's path; "rb" reads raw bytes.
- `src.replace(b"...", b"...")` — b"..." is a bytes literal; replace works on bytes as on text.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `hashlib.sha256(b).hexdigest()` — a fingerprint of bytes, as hex text.
- `json.dumps(d, sort_keys=True) / json.loads(s)` — a dictionary to text and back.
- `open(path, "a")` — append to a file.
- `__enter__ / __exit__` — what a with block calls at its start and end.
- `raise / try ... except` — signal an error / catch one.
- `**d` — a dictionary's entries as keyword arguments.
- `lst[::8], lst[17:22]` — every eighth item; items 18 to 22.

End of Day 173 syntax reference. For the experiments and what they showed, see the Day 173 complete guide.